Universidad de La Habana
Facultad de Matemática y Computación

Proceso de Evaluación Automática en Problemas de Enrutamiento de Vehículos

Autor: **Josué Rodríguez Ramírez**

Tutor: **MSc. Fernando Rodríguez Flores**

Cotutor: **Lic. Rodrigo García Gómez**

Trabajo de Diploma
presentado en opción al título de
Licenciado en Ciencias de la Computación

Noviembre de 2023

A mis padres

Agradecimientos

Le agradezco a Dios, en primer lugar, por escogerme y darme la oportunidad de estudiar esta carrera.

A mi tutor Fernando y mi cotutor Rodrigo por dedicarme parte de su tiempo para desarrollar este trabajo y por su paciencia inagotable.

Agradezco a mis padres y pastores que me han aconsejado, alentado y nunca se cansaron de orar por mí. Al resto de mi familia que me motivaron para llegar hasta aquí.

A mis hermanos en la fe, que oraron incensantemente por esta tesis.

Opinión del tutor

MSc. Fernando Raúl Rodríguez Flores
Facultad de Matemática y Computación
Universidad de la Habana
Noviembre, 2023

Resumen

En este trabajo se propone un método que permite obtener el costo de soluciones vecinas de una solución dada en cualquier VRP, sin tener que implementar un algoritmo para ello. Lo único que se necesita es el código de evaluación de una solución dada para la variante del VRP deseada. El costo de cada solución vecina se calcula de manera eficiente en el sentido de que garantiza que solo se evalúan aquellos fragmentos de la nueva solución en los que esta se diferencia de la solución inicial. El método propuesto garantiza un ahorro en el tiempo de implementación de la solución. El usuario no tendrá que escribir el código para la evaluación eficiente de las soluciones vecinas.

Abstract

In this work, a method is proposed that allows obtaining the cost of neighboring solutions of a given solution in any VRP, without having to implement an algorithm for it. All that is needed is the evaluation code of a given solution for the desired VRP variant. The cost of each neighboring solution is calculated efficiently in the sense that it guarantees that only those fragments of the new solution in which it differs from the initial solution are evaluated. The proposed method guarantees savings in solution implementation time. The user will not have to write code for efficient evaluation of neighboring solutions.

Índice general

Introducción **11**

1. Preliminares **14**
1.1. Problema de Enrutamiento de Vehículos 14
1.2. Algoritmos de Búsqueda Local 16
1.3. Criterios de vecindad 16
1.4. Grafo de Evaluación 17

2. Diferenciación Automática **23**
2.1. Proceso de Diferenciación Automática (AD) 23
2.2. Grafo computacional 25
2.3. Modos de evaluación 25
2.4. Cinta automática 27

3. Desarrollo de la Evaluación Automática **29**
3.1. Tipos de VRP a evaluar 29
3.2. Evaluación de la solución inicial 31
3.3. Obtención y evaluación de las soluciones vecinas 32
3.4. Limitaciones de algunos VRP 33
3.5. Cinta computacional 34

4. Resultados **39**
4.1. VRP con Recogidas y Entregas simultáneas 39
4.2. VRP con Ventanas de Tiempo 41
4.3. Extensibilidad 42

Conclusiones **44**

Recomendaciones **45**

Bibliografía **46**

Índice de figuras

1.1. Variantes del VRP. A: CVRP. B: VRPTW. C: MDVRP. 15
1.2. Grafo inicializado. 19
1.3. Código de evaluación de una solución. La operación `d(x,y)` representa la distancia entre `x` e `y`. 19
1.4. Proceso del grafo de evaluación comenzando con el cliente 1. 20
1.5. Proceso del grafo de evaluación a terminar la ruta A. 20
1.6. Proceso del grafo de evaluación terminado. 21
1.7. Grafo de evaluación transformado al eliminar el cliente 3. . . 21
1.8. Grafo de evaluación transformado al cambiar el cliente 3 de la ruta A a la ruta B. 22

2.1. Grafo de evaluación para el ejemplo, se distinguen los diferentes tipos de variables. 25
2.2. Estructura básica de la cinta. 27

3.1. Grafo de evaluación de un VRP con variables independientes. 30
3.2. Grafo de evaluación de un VRP con rutas disjuntas. Caso donde solo se afecta una subruta. 31
3.3. Grafo de evaluación transformado al eliminar el cliente 3. . . 35
3.4. Grafo de evaluación transformado al eliminar el cliente 3. . . 35
3.5. Cinta de evaluación para el ejemplo 3.3. 36
3.6. Cinta de evaluación para el ejemplo 3.4. 37
3.7. Cinta de evaluación usando el criterio *cambiar un cliente de posición*. 37

4.1. Solución inicial del VRPSPD. 40
4.2. Código de evaluación de una solución de un VRPSPD. La operación `d(x,y)` representa la distancia entre `x` e `y`. 40
4.3. Solución inicial del VRPTW. 41

4.4. Código de evaluación de una solución de un VRPTW. La operación `d(x,y)` representa la distancia entre `x` e `y`. La operación `t(x,y)` representa el tiempo del recorrido entre `x` e `y` 42

Introducción

Los Problemas de Enrutamiento de Vehículos (VRP por sus siglas en inglés) son problemas logísticos en los que las empresas invierten hasta el 20% de sus ganancias [16]. Una optimización de las estrategias de solución de estos problemas significaría un ahorro significativo del presupuesto destinado a resolverlos.

Los VRP son un grupo de problemas de optimización combinatoria que tienen como objetivo determinar un conjunto de rutas para una flota de vehículos que parten de uno o más depósitos para satisfacer las demandas de un grupo de clientes distribuidos en una zona determinada. A su vez, se desea optimizar el costo de algún recurso, ya sea capacidad, combustible, tiempo [6].

El problema original se modifica en dependencia de las demandas de los clientes y a partir de estas modificaciones surgen múltiples variantes como:

- VRP con recogida y entrega (VRPPD) que tiene el objetivo de encontrar una serie de rutas para un conjunto de vehículos que deben suministrar servicio a clientes, con la restricción de que los vehículos tengan suficiente capacidad de transporte para los productos (o personas) que deben ser recogidos y/o entregados a cada cliente (nodo) [14],

- VRP con ventanas de tiempo (VRPTW) donde una flota fija de vehículos de capacidad limitada está disponible en un depósito para atender a un conjunto de clientes con demandas determinadas y cada cliente debe ser visitado dentro de un período de tiempo determinado [7], VRP con más de un depósito (MDVRP) [2, 1],

- VRP con restricciones de capacidad (CVRP) donde cada vehículo tiene una capacidad limitada [1], por solo mencionar algunas.

Cada una de ellas presenta características que la diferencian del resto, por lo que resulta difícil plantear un método genérico que resuelva cualquier VRP.

Estos problemas son NP-duros [11], que son los problemas que no se pueden resolver en un tiempo polinomial. El tiempo y esfuerzo computacional requerido para resolver los VRP aumenta exponencialmente respecto al tamaño del problema. Por esta razón, se deben resolver con métodos aproximados como las heurísticas o metaheurísticas.

Algunos de estos algoritmos son basados en poblaciones como los algoritmos genéticos [9], otros como SA (Simulated Annealing) [8], LNS (Large Neighborhood Search) [12] y VNS (Variable Neighborhood Search) [10] que se basan en una búsqueda local.

Para solucionar un VRP mediante un algoritmo de búsqueda local se parte de una solución inicial S_0 a la que se le hacen transformaciones, como cambiar de rutas, vehículos o clientes y, a partir de estos cambios se obtiene un conjunto de nuevas soluciones que se denomina vecindad de S_0. De este conjunto se selecciona una solución S_1 y a partir de esta última se repite el proceso hasta que se cumpla algún criterio de parada [13].

El problema a resolver es que para evaluar cada una de las nuevas soluciones se debe programar un algoritmo de evaluación, que es diferente en cada VRP. ¿Cómo se puede automatizar el proceso de evaluación de todas las soluciones? ¿Cómo generalizar ese proceso para todos los Problemas de Enrutamiento de Vehículos?

En este trabajo se plantea un método para calcular el costo de las nuevas soluciones para cualquier problema de enrutamiento de vehículos de manera automática. Esto se realiza a través de una estructura denominada *cinta computacional*, es decir, una lista de las transformaciones que se apliquen a la solución inicial.

Esta misma estructura se utiliza en el proceso de diferenciación automática que calcula las derivadas de una función matemática de manera exacta y con la mayor precisión posible en un tiempo razonable que en la mayor parte de los casos prácticos no excede de un pequeño múltiplo del propio tiempo de evaluación de la función [3, 5].

En la Facultad de Matemática y Computación de la Universidad de La Habana se desarrollado métodos de solución generalizados para todas las variante de VRP. Por ejemplo, en [10] se desarrolla una metaheurística de Búsqueda Local (IVNS, por sus siglas en inglés) que permite considerar infinitos criterios de vecindad para un problema, utilizando gramáticas libres del contexto.

Por su parte, en [13] se presenta una forma de calcular el costo de un ve-

cino en casi cualquier variante de VRP, utilizando una estructura llamada Grafo de Evaluación. Algunas variantes, como el Problema de Enrutamiento de Vehículos con Entregas y Recogidas Simultáneas no son solucionadas con esta propuesta. Con la propuesta que se presenta en este trabajo se resuelven todas las variantes de los VRP y extiende la investigación realizada por [13].

Objetivos

El objetivo de este trabajo es desarrollar una estructura que evalúe de manera autómatica las soluciones vecinas de cualquier problema de enrutamiento de vehículos, a partir del grafo de evaluación propuesto en [13]. Para eso se implementa una cinta computacional basada en la idea de la diferenciación automática.

Para lograr los objetivos generales se han trazado los siguientes objetivos específicos:

1. Consultar literatura especializada sobre el estado del arte de los problemas VRP.

2. Estudiar las ideas y códigos relacionados con el Grafo de Evaluación [13]

3. Diseñar e implementar una cinta computacional a partir del Grafo de Evaluación.

4. Analizar los resultados obtenidos en diferentes variantes de VRP.

El presente documento está organizado en 4 capítulos.

En el capítulo 1 se describen las caraterísticas fundamentales de los problemas de enrutamiento de vehículos y los algoritmos de búsqueda local. En el capítulo 2 se describe la estrategia utilizada en la diferenciación automática y cómo aplicarla a los VRP. El capítulo 3 describe el proceso y la estructura implementados. En el capítulo 4 se exponen los resultados alcanzados. Se presentan, además, las conclusiones obtenidas a partir de este trabajo y se brindan algunas recomendaciones para la extensibilidad del sistema en trabajos futuros.

Capítulo 1

Preliminares

En este trabajo se presenta el proceso de evaluación automática de las soluciones vecinas de cualquier VRP.

Este capítulo presenta los conceptos básicos de los problemas de enrutamiento de vehículos y los algoritmos de búsqueda local. Además, se definen los criterios de vecindad que se utilizan en el sistema y la estructura de datos que se utiliza para evaluar las soluciones parciales de manera automática.

La estructura del capítulo es la siguiente: en la sección 1.1 se presentan las características principales de los VRP. En la sección 1.2 se describen los algoritmos de búsqueda local. En la sección 1.3 se explican los criterios de vecindad que se utilizan en el sistema. En la sección 1.4 se describe el grafo de evaluación utilizado para evaluar las soluciones vecinas en cada VRP.

1.1. Problema de Enrutamiento de Vehículos

Desde 1959, cuando George Dantzig y John Ramser introducen la primera definición de un VRP[4], se han desarrollado variantes asociadas a las demandas de los clientes del problema. Estas demandas pueden ser el tiempo de espera, la entrega y recogida de paquetes o la capacidad de los vehículos.

Todos los VRP presentan un depósito inicial, de donde parten los vehículos para satisfacer las demandas de cada cliente. El recorrido termina en el depósito.

A continuación, en 1.1 se presentan esquemas de diferentes variantes de VRP. En la primera variante (CVRP) cada vehículo presenta una capacidad. La segunda variante (VRPWT) muestra clientes con ventanas de tiempo. En

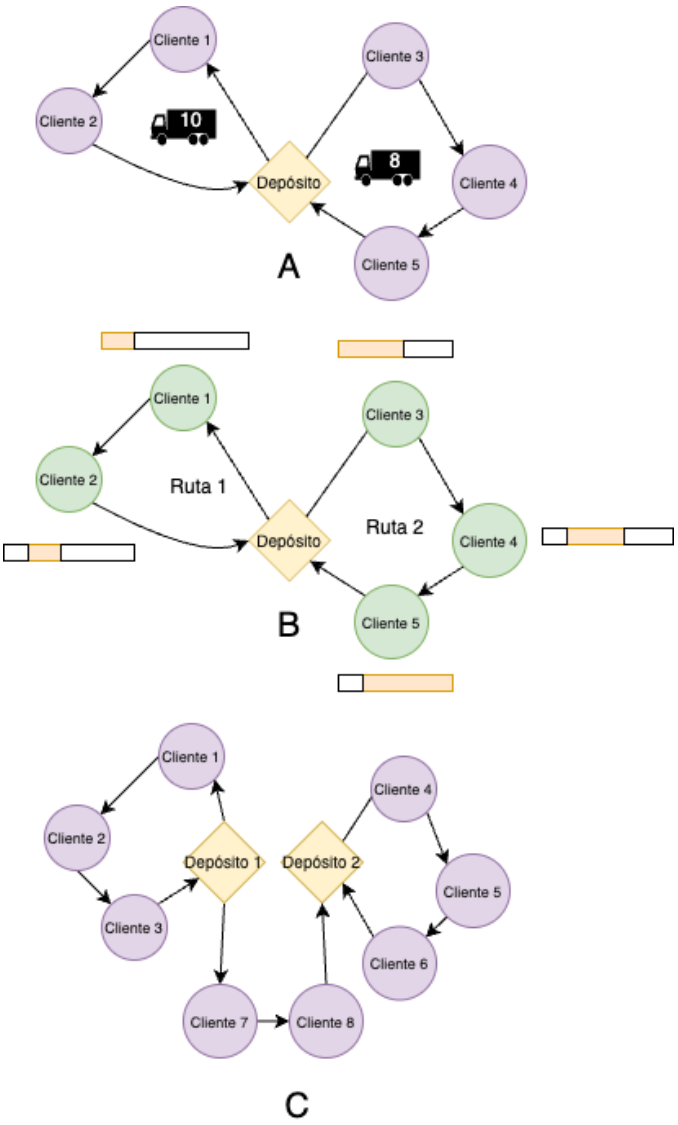

Figura 1.1: Variantes del VRP. A: CVRP. B: VRPTW. C: MDVRP.

la tercera variante (MDVRP) se aprecia más de un depósito para la flota de vehículos.

Como se puede notar, sería muy dificil encontrar un algortimo general que resuelva cualquier VRP, dado que estos problemas presentan restricciones diferentes y, por lo tanto, estrategias diferentes para solucionarlos.

Se han desarrollado heurísticas o metaherísticas que se adpatan a las características específicas de un tipo específico de VRP. En la siguiente sección se presenta la estrategia de búsqueda local en la que se basan un grupo de estas metaherísticas.

1.2. Algoritmos de Búsqueda Local

En los algoritmos de búsqueda local se parte de una solución inicial S_0, que se modifica de varias maneras para obtener un conjunto NS_0 llamado conjunto de soluciones vecinas de S_0, o vecindad de S_0[13]. A las reglas que se aplican para modificar una solución y obtener las soluciones vecinas se les conoce con el nombre de estructura de entorno o criterio de vecindad[13]. Del conjunto de soluciones NS_0 se extrae una nueva solución a modificar. Esto se realiza hasta cumplir con algún criterio de parada.

Existen diferentes algoritmos que se basan en la búsqueda local. Por ejemplo, Enfriamiento Simulado[8], Búsqueda Tabú[15], Búsqueda de Vecindad Variable[10]. Estos algoritmos difieren en los criterios utilizados para obtener las soluciones vecinas. La siguiente sección trata sobre esos criterios de vecindad.

1.3. Criterios de vecindad

Los criterios de vecindad son reglas que permiten transformar una solución de un VRP para obtener nuevas soluciones. Estos criterios pueden ser *"cambiar un cliente de posición"*, *"intercambiar dos clientes"*, o *"cambiar el vehículo de una ruta"*.

Estos criterios se pueden describir como de operaciones elementales. Algunas de estas pueden ser:

1. Seleccionar una ruta `r`.
2. seleccionar un cliente `c` de una posición `p` de una ruta `r`.
3. insertar un cliente `c` en una posición `p` de una ruta `r`.

Por ejemplo, el criterio *cambiar un cliente de posición* está compuesto por las operaciones *extraer cliente de una posición dada* e *insertar al cliente en otra posición*, mientras que el criterio *cambiar el vehículo de una ruta* consta solamente de la operación *asignar a una ruta dada un determinado vehículo disponible*.

Los criterios de vecindad utilizados en un algoritmo de búsqueda local dependen de la variante de VRP que se esté resolviendo [**?**]. Por ejemplo, en un problema donde la flota de vehículos es homogénea pero existen varios depósitos, la operación *cambiar vehículo* no tiene sentido pero *cambiar depósito* sí puede aplicarse.

Para evaluar cada una de las soluciones vecinas del VRP y aplicar los criterios de vecindad se utiliza una estructura de datos llamada grafo de evaluación que se define en la siguiente sección. Este grafo se usa para evaluar de manera automática la solución del problema, es decir, que no será necesario escribir el código de la evaluación.

1.4. Grafo de Evaluación

A partir de la solución inicial del VRP se crea un grafo que se utilizará para evaluar las soluciones vecinas obtenidas por los criterios de vecindad en cada problema del VRP.

En [13] se define formalmente la estructura del grafo de evaluación.

Definición 1 *Entidad del problema: Cada uno de los elementos que componen la solución de una determinada variante de un VRP.*

Por ejemplo, en un problema con flota heterogénea las entidades del problema serían los clientes, los vehículos y el depósito central. En este trabajo se presentan como *variables iniciales*.

Definición 2 *Información del problema: Dato que forma parte de la entrada del problema.*

Un ejemplo de información en el CVRP es la demanda de cada cliente y también la distancia entre cada cliente. Para el HFVRP, la capacidad de cada vehículo es también una información del problema.

Definición 3 *Variable: Se refiere a toda variable que se use durante la evaluación de una solución y contenga un valor numérico que no sea información del problema ni entidad del problema.*

Ejemplos de variables serían *distancia-total*, *demanda-de-ruta*, etc. En este trabajo se presentan como variables intermedias.

También se presentan definiciones para los nodos operacionales. En este trabajo, los nodos operacionales son parte de las variables intermedias, y se les asocia la operación, y los métodos de evaluación y desevaluación, etc.

Definición 4 *Operación: Función que se aplica durante la evaluación de una solución.*

Durante la evaluación de una determinada solución es común aplicar operaciones como *incrementar distancia total, aumentar o disminuir la carga de un vehículo, penalizar exceso de carga,* etc.

Definición 5 *Nodo operacional: Nodo que representa una operación.*

Un nodo operacional es la representación en el grafo de evaluación de una operación. Toda operación recibe una entrada y provoca que se transforme una determinada salida. Por ejemplo, cuando se aplica la operación *incrementa la distancia total en la distancia entre los clientes 1 y 2* se incrementa el valor contenido en la variable *distancia-total* (salida) en dependencia de la distancia entre los clientes 1 y 2 (entradas).

En la definición 5 se establece la representación de una operación, por tanto se hace necesario definir la representación de las entradas y las salidas de dichas operaciones.

Definición 6 *Nodo de entrada/salida: Nodo que representa la entrada o la salida de alguna operación.*

Construcción del grafo de evaluación

Para construir este grafo, en primer lugar, se realiza un proceso de inicialización, donde se construyen únicamente los nodos principales de dicho grafo, que son los correspondientes a cada cliente y depósito. A continuación se muestra un ejemplo del grafo inicializado.

En el ejemplo se presentan 4 clientes (1,2,3,4) y dos rutas (A,B). Un vehículo comienza en un depósito y recorre a los clientes 1, 2 y 3 que se encuentran en A y regresa al depósito, otro vehículo visita al cliente 4 que pertenece a la ruta B:

$$A = [1,2,3] B = [4]$$

En la figura 1.2 se muestra cómo se inicializa el grafo.

Un grafo inicializado no contiene ninguna arista debido a que en un grafo de evaluación las aristas indican las entradas y las salidas de las operacio- nes que se realizan en la evaluación de la solución[13].

Luego de inicializado el grafo se crea el resto del grafo de evaluación a partir de la evaluación de la solución. En la figura se presenta el código que evalúa una solución del VRP.

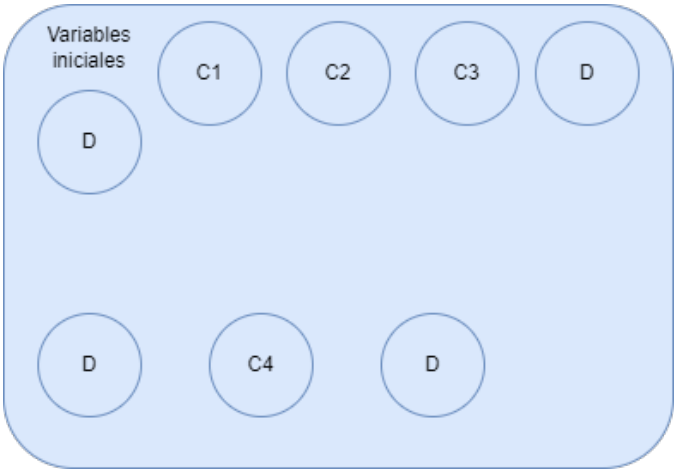

Figura 1.2: Grafo inicializado.

```
evalúa(S):
  new_var distancia_total = 0
  para toda ruta r de S:
      new_var demanda_ruta = 0
      new_var cliente_previo = deposito(r)
      para todo cliente c en r:
          incrementa distancia_total con...
d(cliente_previo, c)
          incrementa demanda_ruta con demanda del cliente c
          cliente_previo = c
      incrementa distancia_total con...
d(c, depósito)
      incrementa distancia_total con...
penalizar(demanda_ruta, capacidad_vehículo)
   devuelve distancia_total
```

Figura 1.3: Código de evaluación de una solución. La operación `d(x,y)` representa la distancia entre `x` e `y`.

En ese proceso de evaluación se realizan operaciones para obtener el costo de la solución. Estos costos pueden ser la distancia recorrida, la demanda de la ruta, la penalización de la ruta, etc. Para guardar el resultado

de estas operaciones se crean nuevos nodos, que llamaremos nodos intermedios.

A continuación se muestra el grafo de evaluación de la primera solución del VRP.

En la figura 1.4 se muestra la creación de una variable intermedia asociada al cliente 1 para guardar el costo de la ruta del depósito al cliente 1. Además, se crea un nodo para guardar la penalización de la ruta y un nodo para guardar la suma total de la ruta 1.

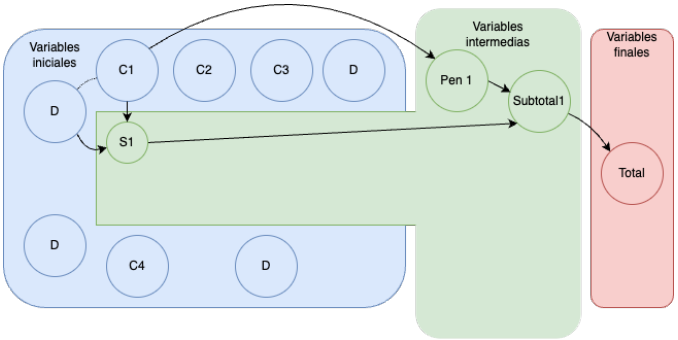

Figura 1.4: Proceso del grafo de evaluación comenzando con el cliente 1.

En la figura 1.5 se muestra que la evaluación de la ruta 1 está calculada.

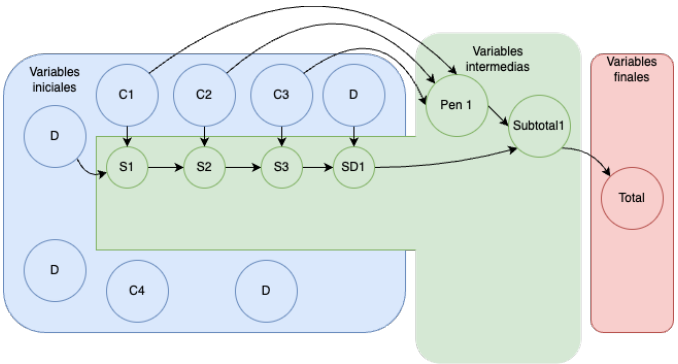

Figura 1.5: Proceso del grafo de evaluación a terminar la ruta A.

Por último, en la figura 1.6 se presenta el grafo de evaluación para esta solución.

De esta manera se puede comprobar que el grafo ha guardado cada cálculo parcial en la evaluación de la solución en las variables intermedias

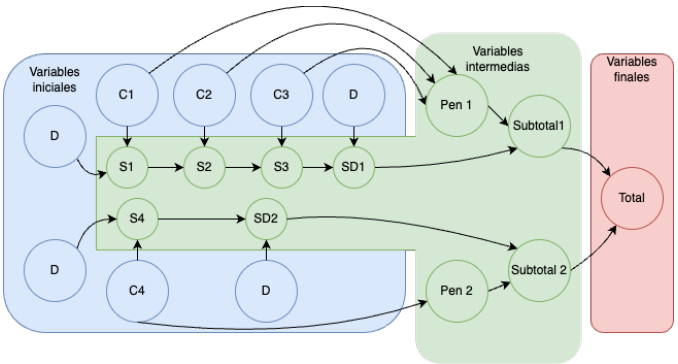

Figura 1.6: Proceso del grafo de evaluación terminado.

creadas, de tal manera que, al aplicar un criterio de vecindad a la solución puede evaluarse sin necesidad de hacer el proceso completo nuevamente. Si a la solución anterior se aplicara, por ejemplo, la operación eliminar cliente C3 como se muestra en la figura 1.7 entonces no se modifica el valor del calculo realizado antes de visitar este cliente o las de otra ruta.

Una nueva solución que se obtenga al aplicar algún criterio de vecindad puede evaluarse a partir de la anterior de manera automática, sin necesidad de escribir un nuevo códgo de evaluación. En el ejemplo que se muestra se pasa de la suma parcial del cliente 2 a la suma parcial del depósito.

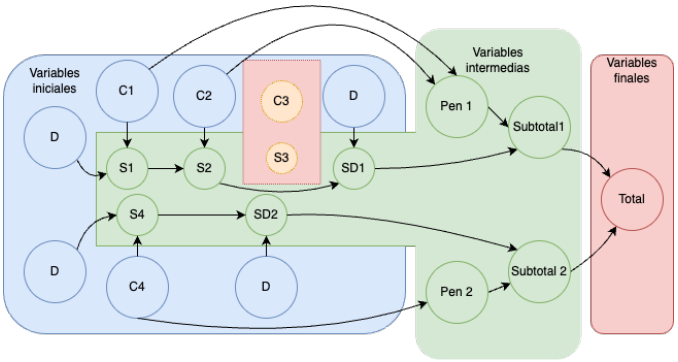

Figura 1.7: Grafo de evaluación transformado al eliminar el cliente 3.

Otro ejemplo sería eliminar este cliente C3 en la ruta A e insertarlo en la ruta B como se presenta en la figura 1.8. En este caso, en la ruta A se evalúa sólo a partir de la suma parcial 2 como en el ejemplo anterior, ya que no se modifica el valor calculado previamente. Además, debe evaluarse la ruta B

a partir del cliente insertado porque el valor calculado de la ruta hasta el cliente anterior no se modificó.

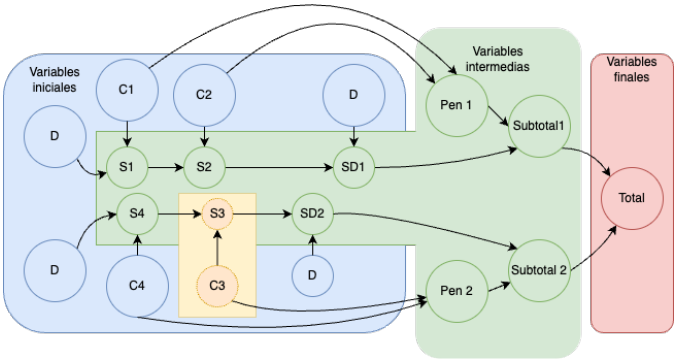

Figura 1.8: Grafo de evaluación transformado al cambiar el cliente 3 de la ruta A a la ruta B.

Los nodos intermedios hacen función de variables acumuladoras, porque guardan el costo de cumplir la demanda de cada cliente y las aristas describen el siguiente nodo a evaluar que depende del valor de nodo anterior.

El proceso de calcular a partir de un nodo o evaluar la ruta desde el depósito hasta el nodo final, depende de una cinta computacional que se conecta a este grafo de evaluación. La idea de esta cinta surge de la Diferenciación Automática que se explica en el siguiente capítulo.

Capítulo 2

Diferenciación Automática

Para obtener la estructura automática que evalúe cada variante de VRP así como los problemas vecinos que se obtienen al aplicarse los criterios de vecindad sin necesidad de reestructurar el código se utiliza una cinta computacional que se aplica, de manera similar, en el proceso matemático de Diferenciación automática y se explica en este capítulo.

Este capítulo está dividido en 5 secciones. La sección 2.1 trata sobre el concepto de diferenciación automática. La sección 2.3 presenta y describe los dos modos de evaluación de esta estrategia. La sección 2.4 presenta la estructura de la cinta de evaluación que utiliza AD.

2.1. Proceso de Diferenciación Automática (AD)

En esta sección se presenta el concepto y las caraterísticas de la Diferenciación Automática.

AD es una estrategia que se utiliza para evaluar la derivada de una función matemática cuya evaluación se realiza a través de un código computacional. Para esto se utiliza la regla de la cadena y se logra obtener la evaluación de las derivadas sin errores numéricos.

La información durante el proceso de evaluación se guarda en variables intermedias, dentro de una cinta de evaluación, que no es más que el registro de una corrida en particular de un programa dado, con valores particulares especificados para las variables de entrada, mostrando la secuencia de valores intermedios calculados y las operaciones realizadas. A esta cinta de evaluación también se le conoce con el nombre de procedimiento de evaluación. En caso de una reestructuración de la función inicial, se mantendrá guardada la información de las partes no alteradas de la función.

Cuadro 2.1 Cinta para la evaluación de la función del ejemplo

$v_{-1} = x_1$
$v_0 = x_2$
$v_1 = tan(v_{-1})$
$v_2 = v_{-1}/v_0$
$v_3 = v_1 * v_2$
$v_4 = sen(v_0)$
$v_5 = cos(v_{-1})$
$v_6 = v_4 * v_5$
$v_7 = v_3 + v_6$
$y = v_7$

Estas partes no modificadas de la función no se vuelven a evaluar y de esta manera no se repeten cálculos innecesarios. A continuación se muestra un ejemplo de cómo se usa esta estrategia de evaluación:

Sea la función $y = f(x1, x2)$ donde:

$$y = tan(x1) * \frac{x1}{x2} + sen(x2) * cos(x1)$$

Se quiere calcular el valor de y en algún punto (x_1^*, x_2^*) La manera en la que se evalúa con esta estrategia es como se presenta en el Cuadro 2.1 donde se muestra una secuencia de variables, las primeras son variables de entrada; luego siguen las intermedias (denotadas por V_i con $i > 0$),y al final las de salida (en este caso es y). Cada variable es calculada a partir de aquellas definidas previamente aplicando una de las operaciones o funciones elementales: suma, resta, multiplicación, seno, exp, etc.

El procedimiento de evaluación no se corresponde directamente con la expresión de f como fue escrita originalmente, sino con una expresión cuya evaluación es más eficiente mediante sub-expresiones repetidas, evaluadas una vez, y luego reutilizadas. El procedimiento de evaluación captura la secuencia precisa de operaciones y argumentos usados por una implementación particular de un algoritmo o función. Las expresiones generadas serán reutilizadas para la evaluación de las derivadas.

Este procedimiento utiliza un grafo computacional que se presenta en la siguiente sección.

2.2. Grafo computacional

La relación entre las variables, es decir, las dependientes e independientes se presentan en un grafo computacional, parecido al utilizado en el sistema de VRP. Para el ejemplo mostrado en esta sección aquí se tiene la Figura 2.1.

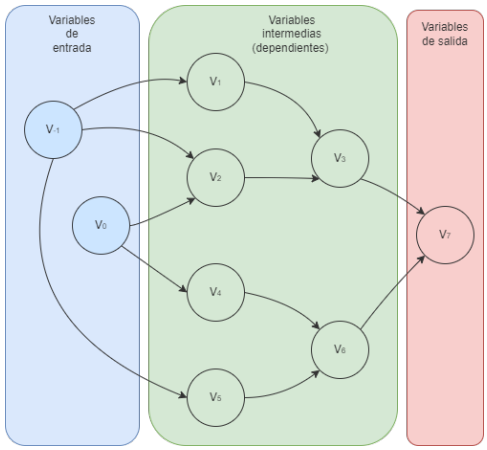

Figura 2.1: Grafo de evaluación para el ejemplo, se distinguen los diferentes tipos de variables.

.

Como se puede notar las raíces del grafo son las variables independientes mientras que los demás nodos son variables dependientes. La diferenciación automática utiliza dos maneras básicas de propagar las derivadas a través de la cinta y así llevar la información precalculada a los siguientes niveles para su uso. Uno es el modo forward y el otro el modo reverse que serán tratados en la siguinte sección.

2.3. Modos de evaluación

El modo forward o hacia adelante se basa en propagar las derivadas desde las variables independientes o de entrada, y a través de las variables intermedias hasta las variables dependientes o de salida del programa, siguiendo el flujo de ejecución del mismo.

De forma general, se aplica la regla de la cadena para cada variable intermedia v_i que se calcula, almacenando los valores obtenidos en nuevas variables v_i^*, que a la vez se utilizan en las subsiguientes aplicaciones de la

regla de la cadena a lo largo de la ejecución del programa, hasta propagar las derivadas a las variables dependientes.

Para mostrar cómo sería el modo forward se toma el ejemplo anterior. El resultado se muestra en el cuadro 2.2.

Cuadro 2.2 Procedimiento de evaluación de la derivada con el modo forward.

$v_{-1} = x_1$
$v^*_{-1} = x^*_1$
$v_0 = x_2$
$v^*_0 = x^*_2$
$v_1 = tan(v_{-1})$
$v^*_1 = 1/cos^2(v_{-1}) * v^*_{-1}$
$v_2 = v_{-1}/v_0$
$v^*_2 = (v^*_{-1} * v_0 - v^*_0 * v_{-1})/v_0^2$
$v_3 = v_1 * v_2$
$v^*_3 = v^*_1 * v_2 + v^*_2 * v_1$
$v_4 = sen(v_0)$
$v^*_4 = cos(v_0) * v^*_0$
$v_5 = cos(v_{-1})$
$v^*_5 = -sen(v_{-1}) * v^*_{-1}$
$v_6 = v_4 * v_5$
$v^*_6 = v^*_4 * v_5 + v^*_5 * v_4$
$v_7 = v_3 + v_6$
$v^*_7 = v^*_3 + v^*_6$
$y = v_7$
$y^* = v^*_7$

Cada v_i representa las variables vistas en el Cuadro 2.1 y cada v^*_i representa su derivada que será propagada.

De la tabla 2.1 a la 2.2 se realiza un procedimiento completamente mecánico. Cada expresión original es sucedida por una correspondiente expresión derivada, de manera que la longitud del procedimiento de evaluación derivado es el doble del original.

El modo reverse o hacia atrás, una vez ejecutado el procedimiento de

evaluación de la función hacia adelante, la información de las derivadas se propaga en sentido contrario al flujo de ejecución del programa, desde las variables dependientes o de salida, y a través de las variables intermedias hasta las variables independientes o de entrada.

En ambos casos se utiliza una cinta automática, que se explica en la siguiente sección.

2.4. Cinta automática

Esta estructura se divide como el mismo grafo de la figura **??**.

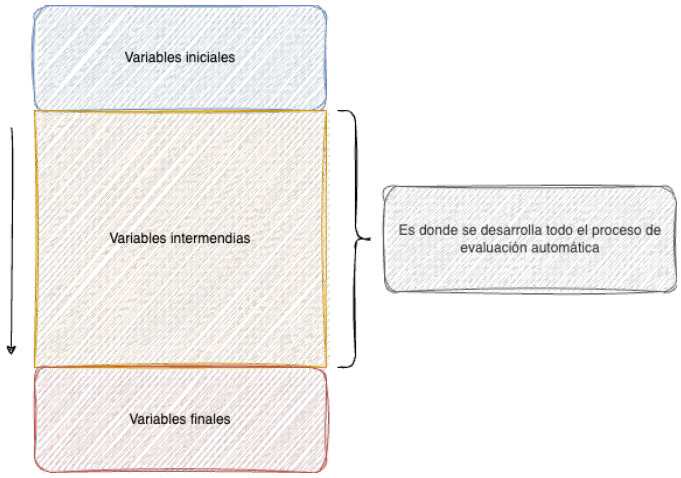

Figura 2.2: Estructura básica de la cinta.

Para uilizar esta cinta en un problema como los problemas de enrutamiento de vehículos se debe definir, en primer lugar, las operaciones básicas (o reglas) que aparecerán en cada uno de los problemas. Como se presentan en el capítulo 1, sección 1.3, se deben definir los criterios de vecindad que modificarán la solución.

Una vez definidas las operaciones se pueden usar en la cinta como ocurre con la diferenciación automática. Es interesante notar cómo se propaga la modificación de cada operación solo por las ramas que se afectan en cada variante del problema y no se afecta el sistema completo, sino solo ese subproblema, que se debe reevaluar.

En este capítulo se presentó el procedimiento de diferenciación automática que utiliza además de un grafo de evaluación, una cinta automática para obtener la derivada de una función matemática.

En el siguiente capítulo se desarrolla esta idea de la diferenciación automática en la evaluación de soluciones vecinas de cualquier VRP.

Capítulo 3

Desarrollo de la Evaluación Automática

En este capítulo se presenta el desarrollo de los métodos que se ejecutan en la evaluación automática, y la incorporación de la cinta computacional al grafo de evaluación.

En la sección 3.1 se presentan los diferentes problemas de enrutamiento de vehículos que se pueden presentar para evaluar. En la sección 3.2 se presenta cómo se realiza la evaluación de la solución inicial. En la sección 3.3 se presenta cómo se modifica este grafo y cómo se realiza la evaluación eficiente de las soluciones vecinas del VRP. En la sección 3.4 se presentan algunas variantes que no pueden aplicar la evaluación automática solamente con el grafo de evaluación. En la sección 3.5 se presenta la incorporación de la cinta al grafo de evaluación.

3.1. Tipos de VRP a evaluar

En esta sección se presentan los diferentes tipos de variantes de VRP que pueden presentarse para solucionar.

Existen muchas variantes de VRP y es difícil garantizar que para cualquiera de ellos solo una parte de ese problema se afectará al aplicarle un criterio. Se puede demostrar que existen cuatro posibilidades para la evaluación automática de un sistema de enrutamiento de vehículos.

Definición 1: *Llamaremos VRP con clientes independientes a aquellos problemas donde las variables de evaluación o restricción de un cliente (ya sea tiempo, combustible, dinero, distancia, capacidad, etc) no afectan a otro cliente. En ese caso, los clientes son independientes.*

Este tipo de problemas tiene características especiales en el grafo de evaluación. Cada nodo auxiliar provee directamente al sumador parcial de la ruta y no a otro nodo auxiliar. Se puede notar en la figura 3.1.

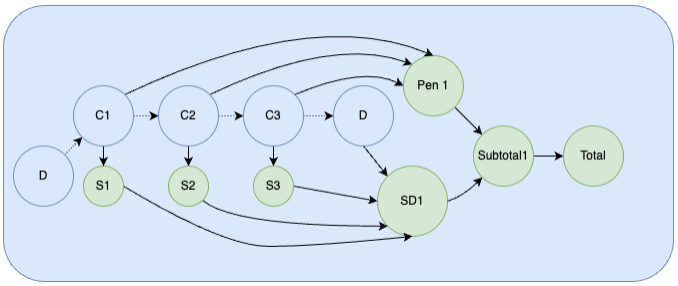

Figura 3.1: Grafo de evaluación de un VRP con variables independientes.

Sobre ese tipo de problemas se tiene el siguiente teorema:

Teorema 1: *En un VRP con clientes independientes, al transformarse un cliente en una ruta solo será necesario reevaluar ese cliente que tributará a la variable de penalización y de suma parcial de la ruta y a la variable del valor final.*

No será necesario reevaluar ningun otro cliente. Al ser independiente del resto no afectará en nada un cambio en su evaluación. Si se agrega o se elimina una cliente, por ejemplo, la evaluación de cada cliente será exactamente igual, no se necesita computar nuevamente la ruta completa. Lo mismo ocurre con rutas disjuntas a la modificada.

Definición 2: *Llamaremos VRP con rutas disjuntas a los problemas donde las rutas son independientes. Es decir, las características de los clientes de una ruta no interfieren o modifican a las de la otra ruta.*

Este tipo de problemas es el más común. La mayoría de los VRP se pueden llevar a convertir en este tipo de problemas. La figura 1.8 muestra un grafo de evaluación de este tipo.

Teorema 2: *En los VRP con rutas disjuntas, al ser modificada una de sus rutas será necesario reevaluar, solamente, la ruta modificada, la evaluación de las rutas restantes será la misma después de la modificación del sistema.*

Como las demás rutas no se ven afectas, entonces, por definición, no será necesario comprobar un cambio en su evaluación anterior, el valor y los componentes seguirán siendo los mismos luego que se modifique dicha ruta.

En estos problemas hay dos casos posibles. En primer lugar, la modificación de un cliente afecta la ruta a partir de él hasta cierto cliente o hasta el final de la ruta, es decir, afecta una subruta. Se muestra este caso en la figura 3.2 donde se afecta desde el cliente 1 hasta el cliente 3, sin embargo, no es necesario volver a evaluar el cliente 4, ya que no se ve afectado.

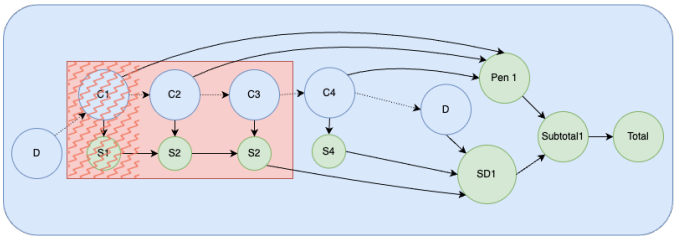

Figura 3.2: Grafo de evaluación de un VRP con rutas disjuntas. Caso donde solo se afecta una subruta.

Los VRP con clientes independientes son un subconjunto de estos problemas. Y estos a su vez son un subconjunto del segundo caso.

El segundo caso ocurre cuando la modificación de un cliente afecta a toda la ruta, y es necesario reevaluarla completa. Por ejemplo cuando cambia el depósito inicial, o las características de un cliente afecta al cliente anterior, que, en consecuencia, se afectan todos.

Definición 3: *Llamaremos VRP con rutas mezcladas a los problemas donde las rutas no son todas independientes.*

En este caso al trasformarse una ruta que no es disjunta se debe reevaluar todo el sistema teniendo un costo igual a la primera evaluación.

3.2. Evaluación de la solución inicial

En el capítulo 1 se mostró cómo se obtenía el grafo de evaluación a partir de la solución inicial. En esta sección se describe cómo se alcanza la evaluación de esa solución inicial.

En cada nodo principal del grafo de evaluación que representa una entidad del problema se puede acceder a la información del problema que es relevante a esa entidad [13]. Es decir, cada nodo que represente a algún cliente contiene la distancia hacia los demás clientes, las demandas y otra información que sea importante para el cliente. Lo mismo ocurre con el resto de los nodos principales.

A los nodos intermedios creados en el proceso de evaluación se le asocia la operación a realizar sobre las demandas de los clientes y dos métodos, uno para evaluar, y otro para desevaluar que deshace la evaluación realizada.

Por ejemplo, si la operación es incrementar la demanda de la ruta, entonces el método evaluar del nodo intermedio asociado a cada cliente suma la demanda del cliente y el valor obtenido del nodo intermedio anterior. El método desevaluar se utiliza para cancelar el método de evaluación de un nodo intermedio.

Al final de cada ruta, el nodo intermedio asociado al último cliente contiene el valor de la evaluación de toda la ruta.

Además de esos nodos intermedios, se crea un nodo penaliza que guarde en cuánto se incumple la demanda de cada cliente en la solución evaluada. En otro nodo intermedio se guarda la evaluación de la ruta una vez penalizada. El valor final de la evaluación de la solución se obtiene a partir de las evaluaciones penalizadas de cada ruta.

Una vez obtenida la evaluación de la solución inicial está construido totalmente el grafo de evaluación que permitirá obtener soluciones vecinas que serán evaluadas automáticamente. Esto se muestra en la siguiente sección.

3.3. Obtención y evaluación de las soluciones vecinas

Si se realizan transformaciones a la solución inicial pueden obtenerse soluciones vecinas que se evalúan automáticamente utilizando el grafo de evaluación. En esta sección se presenta cómo se evalúan estas soluciones y una propuesta para que funcione en cualquier VRP.

En el capítulo 1 se mostró cómo se modifica el grafo de evaluación al aplicar una operación sobre la solución inicial. Cada operación es un método que transforma el grafo de evaluación y evalúa o desevalúa los nodos intermedios afectados por la modificación.

Por ejemplo, en la figura 1.7 se aplica la transformación eliminar cliente. Este es un método que recibe las coordenadas del cliente, es decir, el índice de la ruta en la que se encuentra y su posición en la ruta. El nodo intermedio asociado a ese cliente se desevalúa y se elimina, se conecta el nodo intermedio anterior a él con el nodo intermedio siguiente y se evalúan los nodos intermedios afectados. En el ejemplo de la figura, se deben evaluar todos los nodos intermedios siguientes al nodo eliminado.

En [13] el autor propone almacenar el nodo extraido en una lista de clientes extraidos.

Los métodos que realizan las operaciones de modificación de una solución son genéricas, es decir, para cualquier VRP que se desee resolver, debido a que los métodos de evaluación y desevaluación reciben la operación a partir del nodo intermedio y, las demandas, a partir de los clientes.

Un criterio de vecindad es un conjunto de operaciones que modifican la solución inicial, como se plantea en 1. Al aplicar los métodos de cada una de las operaciones que componen el criterio de vecindad que se utilice se obtiene la evaluación de la nueva solución. Solamente se reevalúan los nodos del grafo que se modificaron por la aplicación de dichas operaciones. Esta evaluación automática se realiza sin repetir cálculo innecesario.

Este proceso de evaluación automática, como se ha presentado hasta aquí, no funciona para todas las variantes de VRP. En la siguiente sección se presentan algunas de estas variantes.

3.4. Limitaciones de algunos VRP

En esta sección se presentan algunas variantes que no pueden utilizar solamente el grafo de evaluación en la evaluación de sus soluciones vecinas.

En algunos problemas de enrutamiento de vehículos como, por ejemplo, el Problema de Enrutamiento de Vehículos con Entregas y Recogidas Simultáneas o el Problema de Enrutamiento de Vehículos con Ventanas de Tiempo, es necesario evaluar otros nodos del grafo de evaluación que no fueron modificados en la aplicación del criterio de vecindad.

En el Problema de Enrutamiento de Vehículos con Entregas y Recogidas Simultáneas, cada cliente recibe y devuelve una cierta cantidad de producto al mismo tiempo. En esta variante, cada cliente es visitado una única vez, lo que da nombre a este VRP. Esto puede llevar a la violación de la restricción de capacidad, ya que la cantidad total de producto que se recoge y se entrega a cada cliente en la ruta puede exceder la capacidad del vehículo, debido a la recogida de productos mientras se atiende a un cliente durante el recorrido. Por tanto, es necesario penalizar el exceso de carga al visitar cada cliente de la ruta.

Si el proceso de penalización se realiza al final de ruta, como ocurre en CVRP (VRP con restricción de capacidad) se puede transformar la solución y reevaluarse hasta el final sin problemas. Sin embargo, si ocurre como en el caso anterior, que la penalización ocurre durante el recorrido, al realizar

una modificación de la solución debe verificarse que el cliente modificado participó o no en la penalización de la ruta.

Lo mismo ocurre en el Problema de Enrutamiento de Vehículos con Ventanas de Tiempo, que consiste en que cada cliente tiene un rango de tiempo para satisfacer su demanda. La penalización ocurre cuando el vehículo no llega en ese rango de tiempo que el cliente presenta. Como esta penalización ocurre durante el recorrido y no al final de la ruta, se debe poder verificar en la modificación de la solución, si el cliente afectó o no la variable penalizar.

En la siguiente sección se muestra la propuesta de solución a esta limitante.

3.5. Cinta computacional

En esta sección se plantean la propuesta de solución que resuelve las limitaciones de algunos VRP frente a la evaluación automática.

El problema que se presenta se basa en la modificación de varibles que son utilizadas en la evaluación final del recorrido o ruta. Tal es el caso de la penalización.

La propuesta que se presenta es utilizar la cinta que utiliza la diferenciación automática, que se desarrolla en el capítulo 2. Esta cinta regula las operaciones de modificación sobre la solución inicial.

En primer lugar, se debe definir cada operación que forman parte de los criterios de vecindad que se aplican a la solución para obtener soluciones vecinas. Una vez que se tengan estas operaciones se devuelve la parte del sistema afectada por esa operación al aplicarla sobre una parte de la solución, y de esta manera, se puede reevaluar esa parte de la solución.

Si se toma el ejemplo representado en la figura 3.3, donde se elimina al cliente 3 de la ruta 1 la operación es *eliminar cliente*. A ese método se le pasan los argumentos *ruta: 1* y *posición: 3*.

A partir de la operación eliminar cliente en la cinta, se verifica la parte de la solución que se modifica y se debe reevaluar. En el caso del ejemplo, se debe reevaluar a partir del cliente anterior hasta el final de la ruta. Para saber si el cliente eliminado afectó la variable penalización este nodo debe ser diferente a los demás nodos intermedios, guardando el valor de penalización de todos los clientes a los que no se les satisfizo la demanda y, por tanto, aumentaron la penalización de la ruta. Por tanto, cuando se elimine al cliente C_3 se reevaluará desde el nodo intermedio asociado al nodo del

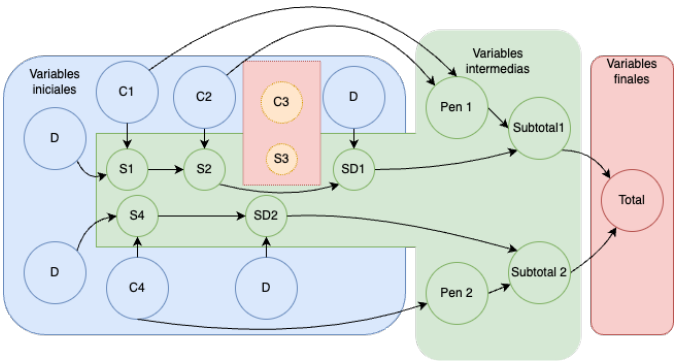

Figura 3.3: Grafo de evaluación transformado al eliminar el cliente 3.

cliente 2 hasta el nodo de salida final, teniendo en cuenta que del nodo penalización se eliminará cualquier penalidad asociada al cliente eliminado.

En el caso de la figura 3.4, donde se inserta el cliente eliminado en la ruta 2, se realiza el mismo proceso de la ruta 1 en la ruta 2.

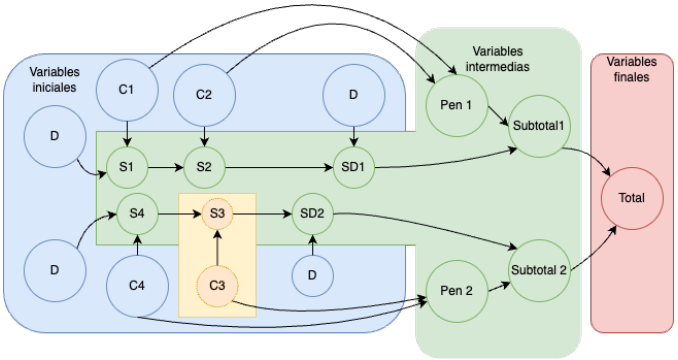

Figura 3.4: Grafo de evaluación transformado al eliminar el cliente 3.

Como la operación es de insertar, entonces, además de evaluar a partir de él, se debe actualizar la variable penalización, en caso de que no se cumpla con la demanda del cliente insertado.

Todas las variables que se modifican en el recorrido en cualquier VRP deben guardar todos los clientes que modifican su valor, como penalización en este caso. Esto hace menos eficiente el algoritmo de evaluación de la solución. En cada variable de ese tipo debe evaluarse el valor de la variable al final de la ruta.

La cinta de evaluación para el ejemplo de la figura 3.3 quedaría de la

siguiente manera:

- Eliminar cliente (1,3)
- Reevaluar (1, 2)

Los parámetros de Eliminar cliente son la ruta y la posición del cliente en la ruta. Los parámetros de reevaluar son la ruta y la posición desde donde se debe evaluar.

La cinta de evaluación para el ejemplo de la figura 3.4 quedaría de la siguiente manera:

- Insertar cliente (2,2)
- Reevaluar (2,1)

Una solución alternativa es evaluar toda la ruta cuando esta contenga alguna varible que se modifica en el recorrido, como penalización. El cliente que modifique alguna variable durante el recorrido de la ruta debe guardarse en la cinta para verificarse en caso de una modificación para encontrar soluciones vecinas.

La cinta de evaluación para el ejemplo de la figura 3.3 con esta propuesta se muestra en la figura 3.5

```
clientes_penalizados
Eliminar cliente (1,3)
por cada cliente_i en clientes_penalizados:
         si cliente_i en ruta 1 e posición cliente_i <
                                        posición cliente 3
            Evaluar ruta 1
            terminar
Reevaluar (1,3)
```

Figura 3.5: Cinta de evaluación para el ejemplo 3.3.

La cinta de evaluación para el ejemplo de la figura 3.4 se muestra en la figura 3.6:

Al utilizar un criterio de vecidad, varias operaciones se desarrollan en la cinta de evaluación. En la figura 3.7 se muestra el criterio *cambiar un cliente de posición*.

```
cliente_i
Insertar cliente (2,2)
por cada cliente_i en clientes_penalizados:
         si cliente_i en ruta 2 e posición cliente_i <
                                        posición cliente 2
            Evaluar ruta 2
            terminar
Reevaluar (2,2)
```

Figura 3.6: Cinta de evaluación para el ejemplo 3.4.

```
clientes_penalizados
Eliminar cliente (1,3)
por cada cliente_i en clientes_penalizados:
         si cliente_i en ruta 1 e posición cliente_i <
                                        posición cliente 3

            Evaluar ruta 1
            terminar
Reevaluar (1,3)
Insertar cliente (2,2)
por cada cliente_i en clientes_penalizados:
         si cliente_i en ruta 2 e posición cliente_i <
                                       posición cliente 2
            Evaluar ruta 2
            terminar
Reevaluar (2,2)
```

Figura 3.7: Cinta de evaluación usando el criterio *cambiar un cliente de posición*.

Esta propuesta realiza más cálculos que las modificaciones que se realizan en la ruta. Pero garantiza que se evalúen los vecinos de cualquier problema de enrutamiento de vehículos.

En este capítulo se explicaron los métodos que se ejecutan en la evaluación automática de las soluciones vecinas, y la incorporación de una cinta computacional al grafo de evaluación para que sea posible usar esta estrategia en cualquier problema de enrutamiento de vehículos. En el siguiente

capítulo se explica cómo se utiliza la evaluación automática en algunas variantes en las que no era posible utilizarse.

Capítulo 4

Resultados

En este capítulo se presentan variantes de problemas de enrutamiento de vehículos a los que no se le podía aplicar la evaluación automática de soluciones vecinas y con la propuesta que se presenta sí la pueden aplicar.

Este capítulo se divide en 3 secciones. La sección 4.2 presenta la evaluación de vecinos del Problema de Enrutamiento de Vehículos con Recogidas y Entregas. La sección **??** presenta la evaluación de vecinos del Problema de Enrutamiento de Vehículos con Ventanas de Tiempo. La sección **??** presenta la extensibilidad de este trabajo.

4.1. VRP con Recogidas y Entregas simultáneas

En esta sección se aplica la evaluación automática a un problema de enrutamiento de vehículos con Recogidas y Entregas simultáneas (VRPSPD). Este problema se explica en el capítulo 3, en la sección 3.4.

La figura 4.3 muestra una solución inicial de un VRPSPD. Este ejemplo presenta dos rutas, con vehículos de capacidad 20 y 15 respectivamente. El primer vehículo sale del depósito con 5 productos y vista los clientes 1, 2 y 3. El segundo vehículo sale del depósito sin productos y visita los clientes 4 y 5. Cada cliente entrega y recibe un cantidad de productos una vez que es visitado. La cantidad que entrega y que recibe se observa en el nodo que representa a cada cliente.

Se deben satisfacer las demandas de cada cliente y obtener la menor distancia para las rutas. El pseudocódigo de la evaluación de la solución inicial se presenta en la figura 4.2.

Las demandas del cliente en este problema son los productos que entrega el cliente menos los productos que recoge.

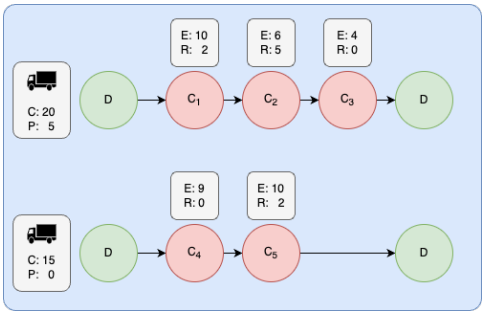

Figura 4.1: Solución inicial del VRPSPD.

```
evalúa(S):
  new_var distancia_total = 0
  para toda ruta r de S:
      new_var demanda_ruta = 0
      new_var penalización = 0
      new_var cliente_previo = deposito(r)
      para todo cliente c en r:
          incrementa distancia_total con...
                     d(cliente_previo, c)
          incrementa demanda_ruta con demandas de c
          si demanda_ruta > capacidad del vehículo...
                            de la ruta r:
              penalizar_en_cinta(c)
              incrementar penalización(
                  demanda_ruta - capacidad...
                  del vehículo de la ruta r)
          cliente_previo = c
      incrementa distancia_total con...
           d(c, depósito)
      incrementa distancia_total con penalización
   devuelve distancia_total
```

Figura 4.2: Código de evaluación de una solución de un VRPSPD. La operación `d(x,y)` representa la distancia entre `x` e `y`.

La función *penalizar_en_cinta* guarda el cliente c en la cinta automática

en caso de que la demanda de la ruta exceda la capacidad del vehículo en el momento en que se visita al cliente c. Al aplicar los criterios de vecindad en la solución se debe comprobar en la cinta los clientes que penalizaron las rutas antes de evaluar la nueva solución.

4.2. VRP con Ventanas de Tiempo

En esta sección se aplica la evaluación automática a un problema de enrutamiento de vehículos con Ventanas de Tiempo (VRPTW). Este problema se explica en el capítulo 3, en la sección 3.4.

La figura 4.3 muestra una solución inicial de un VRPTW. Este ejemplo, igual que el anterior presenta dos ruta. Cada cliente tiene una hora mínima y una hora máxima entre las que puede ser recogido. Si el vehículo no llega entre esas horas significaría una penalización en la ruta.

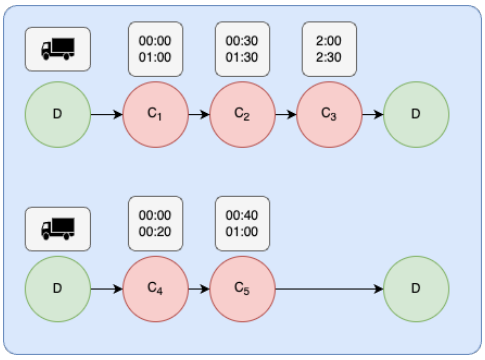

Figura 4.3: Solución inicial del VRPTW.

El pseudocódigo de la evaluación de la solución inicial se presenta en la figura 4.4.

La cinta de evaluación automática guarda los clientes que no fueron recogidos en los tiempos asignados. Al aplicar los diferentes criterios de vecindad para modificar la solución y encontrar vecinos se tienen en cuenta las penalizaciones de los clientes para evaluar toda la ruta o a partir del cliente que se modifica. De esta manera se pueden encontrar soluciones vecinas y evaluarse de manera automática, no solo en estos problemas sino en cualquier VRP, que es el objetivo de este trabajo.

En la siguiente sección se presenta cómo extender la propuesta a otros problemas de enrutamiento de vehículos.

```
1  evalúa(S):
2    new_var distancia_total = 0
3    para toda ruta r de S:
4        new_var tiempo_ruta = 0
         new_var penalización = 0
5        new_var cliente_previo = deposito(r)
6        para todo cliente c en r:
7            incrementa distancia_total con...
                        d(cliente_previo, c)
             incrementa tiempo_ruta con...
                        t(cliente_previo, c)
9            si tiempo_ruta < tiempo mínimo de c
10                penalizar_en_cinta(c)
                  incrementa penalización con...
                     tiempo_ruta - tiempo mínimo de c
             si tiempo_ruta > tiempo máximo de c
10                penalizar_en_cinta(c)
                  incrementa penalización con...
                     tiempo máximo de c - tiempo_ruta

11           cliente_previo = c
12       incrementa distancia_total con...
              d(c, depósito)
         incrementa tiempo_ruta con...
              t(c, depósito)
13       incrementa distancia_total con penalización
14    devuelve distancia_total
```

Figura 4.4: Código de evaluación de una solución de un VRPTW. La operación `d(x,y)` representa la distancia entre `x` e `y`. La operación `t(x,y)` representa el tiempo del recorrido entre `x` e `y`

4.3. Extensibilidad

En esta sección se comentan algunos problemas diferentes a los presentados en la sección anterior y que se le puede aplicar la evaluación automática.

Para cada variante de VRP deben implementarse diferentes opereacio-

nes, no solo mover clientes. Por ejemplo, en el Problema de Enrutamiento de Vehículos con Múltiples Depósitos (MDVRP) se tiene una lista de depósitos y las rutas pueden iniciar y terminar en depósitos distintos. Por ese motivo es necesario implementar las operaciones cambiar el primer depósito de una ruta y cambiar el último depósito de una ruta. Estas operaciones pueden realizarse de la misma manera que las inserciones y extracciones de clientes y el nuevo depósito a insertar se toma de la lista de depósitos disponibles.

Las operaciones implementadas se usan en la cinta computacional para evaluar las soluciones vecinas de la misma manera que se expuso en este capítulo.

A continuación se presentan las conclusiones y recomendaciones del presente trabajo.

Conclusiones

En este estudio, se ha descrito el proceso de evaluación automática de soluciones vecinas en un Problema de Enrutamiento de Vehículos utilizando un grafo de evaluación. Este enfoque permite evaluar eficientemente las soluciones vecinas sin la necesidad de escribir código específico para esta tarea.

A partir de este grafo, es posible obtener el valor de cualquier solución vecina al evaluarla en la función objetivo. Se ha resuelto una limitación de la evaluación automática, la cual impedía su aplicación en problemas que involucren penalizaciones durante el recorrido de una ruta.

Se agregó la cinta computacional que extiende este proceso a cualquier problema de enrutamiento de vehículos. Esta estructura permite conocer la parte de la solución afectada al aplicarle a la solución inicial un criterio de vecindad.

Por último, se ha descrito cómo el método de evaluación automática puede ser extendido a otras variantes del Problema de Enrutamiento de Vehículos, utilizando nuevos criterios de vecindad aplicables a cada una de ellas.

Recomendaciones

Como recomendaciones y trabajos futuros se propone implementar la evaluación automática de soluciones vecinas para variantes de VRP con rutas mezcladas donde no solo sea necesario reevaluar la ruta en la solución vecina, sino todo el sistema. se proponen otras variantes como el Problema de Enrutamiento de Vehículos con Trasbordo.

Se sugiere realizar una investigación sobre la implementación de la evaluación automática de soluciones vecinas en algoritmos genéticos, con el fin de obtener de manera automática el resultado de los cruzamientos entre soluciones y el costo asociado a estos resultados, sin necesidad de programarlo manualmente.

Se recomienda diseñar una biblioteca que pueda generar el código para la evaluación automática en cualquier lenguaje de programación, lo que permitiría que el método propuesto en este trabajo pudiera aplicarse en otras soluciones.

Bibliografía

[1] Gabriel Alemany, Angel Alejandro Juan, Roberto García, Álvaro García, and M. Ortega. Multi-capacity, multi-depot, multi-product vrp with heterogeneous fleets and demand exceeding depot capacity. 2015. (Citado en la página 11).

[2] Bilgesu Bayir, Aurélie Charles, Aicha Sekhari, and Yacine Ouzrout. Multi-depot vehicle routing problem with branch-and-cut: An application in short food supply chains. 2022. (Citado en la página 11).

[3] Manuel A. Ferreiro Caballero. Adolnet: Herramienta para la diferenciación automática en .net. Master's thesis, Facultad de Matemática y Computación, Universidad de La Habana, La Habana, Cuba, 2010. (Citado en la página 12).

[4] George B Dantzig and John H Ramser. The truck dispatching problem. *Management science*, 6(1):80–91, 1959. (Citado en la página 14).

[5] A. Griewank. On automatic differentiation. *Mathematical Programming: Recent Developments and Applications*, 40:83–108, 1989. (Citado en la página 12).

[6] F. S. Hillier and G. Lieberman. *Introducción a la investigación de operaciones*. McGraw-Hill, novena edition, 2006. (Citado en la página 11).

[7] Antoon W. J. Kolen, Alexander H. G. Rinnooy Kan, and Harry W. J. M. Trienekens. Vehicle routing with time windows. *Oper. Res.*, 35:266–273, 1987. (Citado en la página 11).

[8] Wan-hui Liu, Shu-jun Tian, Chun-qiang Jia, and Yu-Ning Cao. A hybrid genetic-simulated annealing algorithm for optimization of hydraulic manifold blocks. *Journal of Shanghai University (English Edition)*, 12(3):261–267, 2008. (Citado en las páginas 12 y 16).

[9] David I. Mester and Olli Bräysy. Active-guided evolution strategies for large-scale capacitated vehicle routing problems. *Comput. Oper. Res.*, 34:2964–2975, 2007. (Citado en la página 12).

[10] Camila Pérez Mosquera. Primeras aproximaciones a la búsqueda de vecindad infinitamente variable. Master's thesis, Facultad de Matemática y Computación, Universidad de La Habana, La Habana, Cuba, May 2017. (Citado en las páginas 12 y 16).

[11] Garey M. R. and D. Johnson. *Computers and intractability, A Guide to the theory of NP- Completeness*. W.H.Freeman and Company, 1979. (Citado en la página 12).

[12] Stefan Røpke and David Pisinger. An adaptive large neighborhood search heuristic for the pickup and delivery problem with time windows. *Transp. Sci.*, 40:455–472, 2006. (Citado en la página 12).

[13] José Jorge Rodríguez Salgado. Una propuesta para la evaluación automática de soluciones vecinas en un problema de enrutamiento de vehículos a partir del grafo de evaluación de una solución. Master's thesis, Facultad de Matemática y Computación, Universidad de La Habana, La Habana, Cuba, 2020. (Citado en las páginas 12, 13, 16, 17, 18, 31 y 33).

[14] Pedro Pablo Ballesteros Silva and Antonio Escobar Zuluaga. Review of state of the art vehicle routing problem with pickup and delivery (vrppd). *Ingeniería y Desarrollo*, 34:463–482, 2016. (Citado en la página 11).

[15] Éric Taillard, Philippe Badeau, Michel Gendreau, François Guertin, and Jean-Yves Potvin. A tabu search heuristic for the vehicle routing problem with soft time windows. *Transportation science*, 31(2):170–186, 1997. (Citado en la página 16).

[16] P. Toth and D. Vigo. *The Vehicle Routing Problem*. Society for Industrial and Applied Mathematics, 2002. (Citado en la página 11).